

Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Motivation	1
Compile-Time vs. Run-Time Work	2
Literature Survey	4
Overview	10
Staging	22
Shape Propagation	29
Benchmarking	58
Bibliography	60

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, general style, even when parts of it could be computed during compilation.

Original program

```
1 fun dot(xs, ys, i) = mls  
2   if xs.length == i then 0  
3   else xs.(i) * ys.(i)  
4       + dot(xs, ys, i + 1)  
5  
6 fun dotWith3(v) =  
7   dot([1, 0, 2], v, 0)
```

can actually be written as

```
1 fun dotWith3(v) = v(0)  
  + 2 * v(2) mls
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matches, the multiplication by 0 — all known at compile time. The residual program contains only the work that genuinely depends on the runtime input v .

Goal: let the programmer keep the abstract version, and have the compiler peel away the static layer automatically.

Motivation	1
Literature Survey	4
Multi-Stage Programming	5
Class specialization	8
Overview	10
Staging	22
Shape Propagation	29
Benchmarking	58
Bibliography	60

Multi-Stage Programming

Well-known optimization technique [1]

```
1 fun power(n, x) = if n is
2   0 then .<1>.
3   else .<~x * ~ (power(n - 1, x))>.
4 fun power2 = .! .<(x => ~ (power(2, .<x>.) ) )>.
```

In here, green annotates code to be executed in the next stage, and gray executed in the current stage.

Multi-Stage Programming

Rule for $.!$: evaluate until the whole code fragment is in the next stage, then move it to the current stage.


$$.!\left(\left(\left(\left(x\right)\right)\right)\right) = .!\left(x\right) = x$$

During evaluation, we are able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

1 `x => power(2, x)`

mls

2 `x => if 2 is 0 then 1 else x * power(2 - 1, x)`

3 `x => x * power(1, x)`

4 `x => x * if 1 is 0 then 1 else x * power(1 - 1, x)`

5 `x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)`

6 `x => x * x * 1`

7 `fun power2 = x => x * x * 1`

Class specialization

```
1 class B(val x) with
2   fun f() = ...
3   // ...
4   if x is
5     1 then new D1(1)
6     2 then new D2(2)
7
8   x.f()
```

```
...           ...
8   if x is mls
9     D1 then x.D1_f1()
10    D2 then x.D2_f1()
11    D3 then x.D3_f1()
```

Traditional Multi-Stage Programming tracks types.

Even if x has a known class shape, we cannot remove matching arms.

Specialization is done through typing, so we know that $x : B$, but we do not know which specific derived class of B is used.

Class specialization

```
1 data class Foo(x, y) extends Bar(x+1, y+1)
2
3 if new Foo(1, 2) is
4   Bar(x, y) then ...
```

mls

Under single inheritance, matching arms runs into problems. [2]

(Class splitting)

```
1 class Foo$1$2 extends Bar
2 class Bar$2$3
```

mls

Motivation 1

Literature Survey 4

Overview 10

 Approach in High Level 11

 MLscript Compiler 14

 Recipe 15

Staging 22

Shape Propagation 29

Benchmarking 58

Bibliography 60

Approach in High Level

Start with an ordinary module:

```
1 module M with
2   fun dot(xs, ys, i) =
3     if xs.length == i then 0
4     else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
5   fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

mls

Approach in High Level

Mark it as staged: the module now acts as a **code generator**: at compile time it specialises calls on known argument shapes and emits a residual module containing only the work that depends on runtime input.

```
1  staged module M with
2    fun dot(xs, ys, i) =
3      if xs.length == i then 0
4      else xs.(i) * ys.(i) + dot(xs, ys, i + 1)
5  fun dotWith3(v) = dot([1, 0, 2], v, 0)
```

mls

Approach in High Level

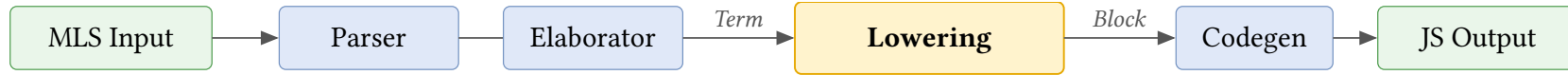
After staging, We are left with an ordinary (non-staged) module containing just the residual program that the user can use

```
1 module M with
2   fun dotWith3(v) = v.(0) + 2 * v.(2)
3   ... with more specialized functions ...
```

mls

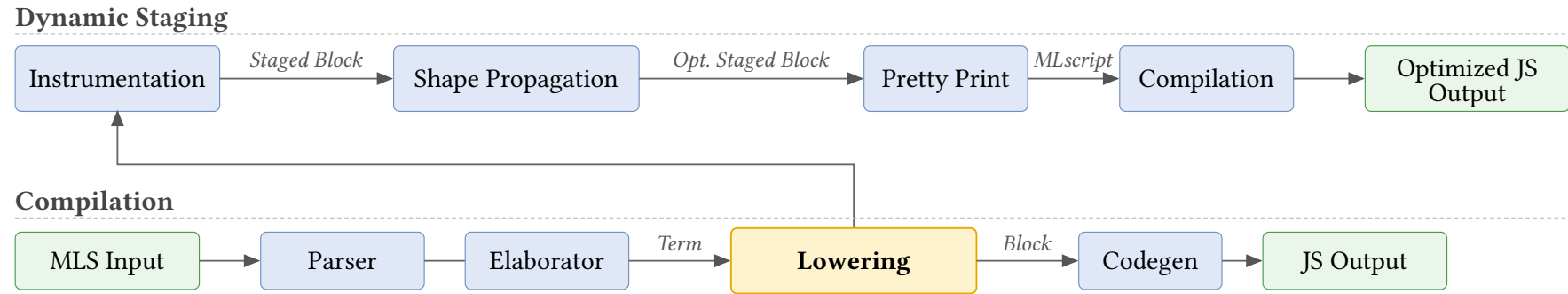
Hence, our approach is a combination of **metaprogramming** (code generator) and **specialization** (generation of specialized functions).

MLscript Compiler



- Lexer/Parser: converts source code into AST
- Elaborator/Resolver: Deal with references
- **Lowering**: converts AST to a simplified format
- Codegen: Turn AST into executable code (JavaScript).

Recipe



Two phases: Instrumentation / Shape Propagation

Lowering Pass

We implement staging a transformation from Scala Block $\Rightarrow$ Scala Block.

Recipe

Dynamic Staging



Compilation



Source (MLscript)

```
if C(0) is
  Int then "Int"
  C(n) then "C(" + n + ")"
  else "Unknown"
```

Recipe

Dynamic Staging



Compilation



Block

```
Scoped([scrut, n, arg, tmp],
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n, Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ""]))
    ),
  ], Ret("Unknown")))
```

Recipe

Dynamic Staging



Compilation



Staged Block

```
Scoped([scrut, n, arg, tmp],
  Assign(scrut, Call(C, [0])),
  Match(Ref(scrut), [
    Arm(Cls(Int), Ret("Int")),
    Arm(Cls(C),
      Assign(arg, Select(scrut, "n")),
      Assign(n, Ref(arg)),
      Assign(tmp, Call("+", ["C(", n])),
      Ret(Call("+", [tmp, ""]))
    ], Ret("Unknown"))
```

Recipe

Dynamic Staging



Compilation



Optimized Staged Block
`Block.Ret("C(0)")`

Recipe

Dynamic Staging



Compilation



MLscript
"C(0)"

Recipe

Dynamic Staging



Compilation



Optimized JS
"C(0)"

Motivation 1

Literature Survey 4

Overview 10

Staging 22

 Staging Symbols 27

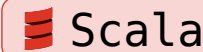
 Instrumentation 28

Shape Propagation 29

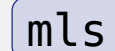
Benchmarking 58

Bibliography 60

```
1 case class Return(res, implct) extends Block
2 case class Match(scrut, arms, dflt, rest) extends Block
3 case class Assign(lhs, rhs, rest) extends Block
```



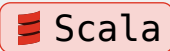
```
1 class Block with
2   constructor
3   Return(res, implct)
4   Match(scrut, arms, dflt, rest)
5   Assign(lhs, rhs, rest)
```



For any Scala Block data, we can recreate the same structure which we called **Staged Block**.


```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  val pow = Symbol("pow"); val x = Symbol("x"); val n = Symbol("n")
2  val t1 = Symbol("tmp"); val t2 = Symbol("tmp")
3  val sub = Symbol("-"); val mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped(HashSet(t1, t2), Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```



As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  let pow = Symbol("pow"); let x = Symbol("x"); let n = Symbol("n")
2  let t1 = Symbol("tmp"); let t2 = Symbol("tmp")
3  let sub = Symbol("-"); let mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped([t1, t2], Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```

mls

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

`ValueLit(lit)`

mls

Example: $1 \mapsto \text{ValueLit}(1)$

`Symbol(name)`

`Symbol(name)`

mls

Example:

- ▶ $x \mapsto \text{Symbol}("x")$
- ▶ $C \mapsto \dots?$

This is not enough for staging symbols. We'll revisit this case later on.

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

`ValueLit(lit)`

mls

Example: $1 \mapsto \text{ValueLit}(1)$

`Value.Ref(sym)`

`let l = sym`

mls

`ValueRef(l)`

Example: $x \mapsto \text{ValueRef}(\text{Symbol}("x"))$

`Select(qual, name)`

`let q = qual`

mls

`let n = name`

`Select(q, n)`

Example: $x.p \mapsto \text{Select}(\text{ValueRef}(\text{Symbol}("x")), \text{Symbol}("p"))$

```
Tuple([x0, x1, ...])
```

```
let s0 = x0
```

mls

```
let s1 = x1
```

...

```
Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging Symbols

- ▶ We need more information about the Symbol in the original stage

Add reference to current value in the symbol

C.x

```
let CSym = ClassSymbol("C", C)
Select(ValueRef(CSym), Symbol("x"))
```

mls

Add redirection within current stage to allow access for next stage

```
staged class A with
  fun f() = M.f()
  val redirect_M = M
```

mls

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For local symbols, we can maintain a map during staging to reuse a staged symbol.

`x + x`

```
let x = Symbol("x")
```

mls

```
// let x1 = Symbol("x")
```

```
Call(ValueRef(Symbol("+")), [[ValueRef(x), ValueRef(x)]])
```

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For class and module symbols, we need to cache and use first instance of a symbol within the staged code.

`M.f()`

```
let MSym = symbolMap.check(ModuleSymbol("M", M))  
Call(Select(ValueRef(MSym), Symbol("f")), [[]])
```

mls

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

```
staged module Math with
  val funCache = new Map()
  val generatorMap = new Map([["pow", pow_gen]])
  fun pow_staged() = ...
  fun pow_gen(x, n) =
    specialize(funCache, "f", pow_staged, [x, n])
  fun propagate() =
    let dyn = ...
    pow_gen(dyn, dyn)
    sq_gen(dyn)
```

mls

Motivation	1
Literature Survey	4
Overview	10
Staging	22
Shape Propagation	29
Shape Definition	30
Shape Propagation (Example)	32
Shape Propagation (Formalization)	34
Shape Propagation (Pattern Matching)	41
Pattern Matching (Formalization)	46
Printing Staged Block	56
Benchmarking	58
Bibliography	60

Shape Definition

We focus on tracking the shape of the Paths and Results.

$$s ::= \iota \mid \mathbf{dyn} \mid [\bar{s}] \mid C(\overline{n:s}) \mid \perp \mid s \cup s$$

```
fun f(p, cond) =
```

mls

```
  let a = 42
```

```
  let b = new C(p, 2)
```

```
  let c = if cond then [1, 2, 3] else C(1, 2)
```

```
  let d = if b is C then 0 else 1
```

Shape Definition

Block in BNF

Symbol x, y, z

Literal ι

Function definition **fun** $f(\overline{x_i^n}) = b$

Plain class definition **class** $C(\overline{\text{val } n})$

Staged module definition **staged module** M **with fun** $f(\overline{x_i^n}) = b$

Path $p ::= \iota \mid x \mid p.n$

Result $r ::= p \mid f(\overline{p}) \mid \text{new } C(\overline{p}) \mid [\overline{p}]$

Match pattern $\pi ::= \iota \mid C \mid []^n \mid _$

Match arm $\kappa ::= \pi \text{ then } b$

Non-tail block $B ::= \varepsilon \mid \text{if } p \text{ is } \overline{\kappa}; B \mid x = r; B \mid \text{let } x; B$

Block $b ::= \text{return } r \mid \text{end} \mid B; b$

Shape $s ::= [\iota] \mid \text{dyn} \mid [[\overline{s}]] \mid [C(\overline{n : s})] \mid \perp \mid s \cup s$

Shape context $\Gamma ::= \varepsilon \mid \Gamma, p \mapsto s \quad (p \neq x)$

Shape Propagation (Example)

A **Shape** captures what we statically know about a value at compile time.

$$s ::= \iota \mid \mathbf{dyn} \mid [\bar{s}] \mid C(\overline{n : s}) \mid \perp \mid s \cup s$$

We write $\llbracket s \rrbracket$ to denote the shape of an expression, distinguishing it from actual values.

```
fun f(p, cond) =
```

mls

```
  let a = 42
```

```
  let b = C(p, 2)
```

```
  let c = if cond then [1, 2] else C(1, 2)
```

```
  let d = if b is C then 0 else 1
```

$\llbracket 42 \rrbracket$

$\llbracket C(\mathbf{dyn}, 2) \rrbracket$

$\llbracket [1, 2] \cup C(1, 2) \rrbracket$

$\llbracket 0 \rrbracket$

Shape Propagation (Example)

A First Taste

A tiny program, propagating shapes by hand:

```
staged module M with
  fun add1(x) = x + 1
  fun test()  = add1(2)
```

mls

Walking through `test()`:

1. Argument 2 has shape $\llbracket 2 \rrbracket$ (a literal).
2. Specialise `add1` with $x \mapsto \llbracket 2 \rrbracket$ inside the body, x looks up $\llbracket 2 \rrbracket$ in the context.
3. $x + 1$ folds: $\llbracket 2 \rrbracket + 1 \rightarrow \llbracket 3 \rrbracket$.
4. Cache the result as `add1_Lit2() = 3`; rewrite the call site `add1(2)` to `add1_Lit2()`.

Final shape of `test()`: $\llbracket 3 \rrbracket$. Body collapses to a constant.

Shape Propagation (Formalization)

Shape of Path ($\Gamma \vdash p \Rightarrow s$)

We evaluate paths p to their shapes s .

$$\frac{}{\Gamma \vdash \iota \Rightarrow \llbracket \iota \rrbracket}$$

Example

42

mls

$\Gamma = \varepsilon$

$\varepsilon \vdash 42 \Rightarrow \llbracket 42 \rrbracket$

$$\frac{(p \mapsto s) \in \Gamma \quad s \neq \perp}{\Gamma \vdash p \Rightarrow s}$$

Example

x

mls

$\Gamma = x \mapsto \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$\Gamma \vdash x \Rightarrow \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$$\frac{p \notin \text{dom}(\Gamma) \quad \Gamma \vdash p \Rightarrow s}{\Gamma \vdash p.n \Rightarrow \text{sel}(s, \llbracket n \rrbracket)}$$

Example

x.n

mls

$\Gamma = x \mapsto \llbracket C(n : \llbracket 1 \rrbracket) \rrbracket$

$\Gamma \vdash x.n \Rightarrow \llbracket 1 \rrbracket$

Reminder: $p ::= \iota \mid x \mid p.n$

Shape Propagation (Formalization)

Shape Selection (**sel**)

sel computes the shape of a selection.

$$\text{sel}(\llbracket C(\overline{n_i : s_i}) \rrbracket, \llbracket n_i \rrbracket) = s_i$$

$$\text{sel}(\llbracket [\overline{s_i}^{i \in 0..n}] \rrbracket, \llbracket i \rrbracket) = s_i$$

$$\text{sel}(s_1, s_2) = \mathbf{err} \quad \text{otherwise}$$

$$\text{sel}(\mathbf{dyn}, \llbracket n_i \rrbracket) = \mathbf{dyn}$$

$$\text{sel}(s_1 \cup s_2, s) = \text{sel}(s_1, s) \cup \text{sel}(s_2, s)$$

Example

$$\begin{aligned} \text{sel}(\llbracket [1, 2] \rrbracket \cup \llbracket [2, 3, 4] \rrbracket, \llbracket 1 \rrbracket) &= \text{sel}(\llbracket [1, 2] \rrbracket, \llbracket 1 \rrbracket) \cup \text{sel}(\llbracket [2, 3, 4] \rrbracket, \llbracket 1 \rrbracket) \\ &= \llbracket 2 \rrbracket \cup \llbracket 3 \rrbracket \end{aligned}$$

Shape Propagation (Formalization)

Shape of Result (sor: $\Gamma \vdash r \rightsquigarrow r' \Rightarrow s$)

We optimize block and track shapes simultaneously: r becomes r' with shape s .

$$\frac{\Gamma \vdash p \Rightarrow s}{\Gamma \vdash p \rightsquigarrow p \Rightarrow s}$$

Example

x

mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket \\ \Gamma &\vdash x \rightsquigarrow x \Rightarrow \llbracket 1 \rrbracket\end{aligned}$$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash [\overline{p_i}^n] \rightsquigarrow [\overline{p_i}^n] \Rightarrow \llbracket [\overline{s_i}^n] \rrbracket}$$

Example

[x, y]

mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket, y \mapsto \llbracket 2 \rrbracket \\ \Gamma &\vdash [x, y] \rightsquigarrow [x, y] \Rightarrow \llbracket \llbracket 1 \rrbracket, \llbracket 2 \rrbracket \rrbracket\end{aligned}$$

$$\frac{\Gamma \vdash p_i \Rightarrow s_i}{\Gamma \vdash \mathbf{new} C(\overline{p_i}) \rightsquigarrow \mathbf{new} C(\overline{p_i}) \Rightarrow \llbracket C(\overline{n_i} : \overline{s_i}) \rrbracket}$$

Example

new Pair(x, y)

mls

$$\begin{aligned}\Gamma &= x \mapsto \llbracket 1 \rrbracket, y \mapsto \llbracket 2 \rrbracket \\ \Gamma &\vdash \mathbf{new} \text{Pair}(x, y) \Rightarrow \llbracket \text{Pair}(a : \llbracket 1 \rrbracket, b : \llbracket 2 \rrbracket) \rrbracket\end{aligned}$$

Reminder: $r ::= p \mid [\overline{p}] \mid \mathbf{new} C(\overline{p}) \mid f(\overline{p})$

Shape Propagation (Formalization)

Shape of Result (sor) – Staged Application

When f is **staged**, we recursively specialise its body on the argument shapes.

$$\frac{f \text{ is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}(\bar{p}, \bar{s}) = (r', s)}{\Gamma \vdash f(\bar{p}_i) \rightsquigarrow r' \Rightarrow s}$$

Example

```
staged module Staged with
  fun pyth(x, y) = x * x + y * y
  fun test() = pyth(2, 3)
```

mls

Specialise $\text{pyth}_{\text{gen}}(\llbracket 2 \rrbracket, \llbracket 3 \rrbracket) \rightarrow \text{pyth_Lit2_Lit3}() = 13$, shape $\llbracket 13 \rrbracket$.

Conclusion: $\varepsilon \vdash \text{pyth}(2, 3) \rightsquigarrow 13 \Rightarrow \llbracket 13 \rrbracket$

Reminder: $r ::= p \mid [\bar{p}] \mid \text{new } C(\bar{p}) \mid f(\bar{p})$

Shape Propagation (Formalization)

Shape of Result (sor) – After Specialisation

After propagation, the call site `pyth(2, 3)` is rewritten to its cached variant.

$$\frac{f \text{ is staged} \quad \Gamma \vdash p_i \Rightarrow s_i \quad f_{\text{gen}}(\bar{p}, \bar{s}) = (r', s)}{\Gamma \vdash f(\bar{p}_i) \rightsquigarrow r' \Rightarrow s}$$

Example

```
staged module Staged with
  fun pyth_Lit2_Lit3() = 13
  fun test() = pyth_Lit2_Lit3()
```

mls

Reminder: $r ::= p \mid [\bar{p}] \mid \text{new } C(\bar{p}) \mid f(\bar{p})$

Shape Propagation (Formalization)

Shape of Result (sor) – Non-Staged Application

When f is **non-staged**, we cannot peek into its body. Two sub-cases.

$$\frac{\begin{array}{l} f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \\ f_{\text{imp}} \left(\overline{\text{valOf}(s)} \right) = (r, s) \end{array}}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow r \Rightarrow s}$$

$$\frac{\begin{array}{l} f \text{ is non-staged} \quad \Gamma \vdash p_i \Rightarrow s_i \\ \exists i. \neg \text{static}(s_i) \end{array}}{\Gamma \vdash f(\overline{p_i}) \rightsquigarrow f(\overline{p_i}) \Rightarrow \mathbf{dyn}}$$

Example

```
// fun sq(x) = x * x (non-  
staged, pure)
```

mls

```
NonStaged.sq(2)
```

All args static; evaluate $\text{sq}(2) = 4$.

$\varepsilon \vdash \text{sq}(2) \rightsquigarrow 4 \Rightarrow \llbracket 4 \rrbracket$

Example

```
// pyth's body, x has dynamic  
shape
```

mls

```
NonStaged.sq(x)
```

$\Gamma = x \mapsto \mathbf{dyn}$; cannot evaluate, emit code.

$\Gamma \vdash \text{sq}(x) \rightsquigarrow \text{sq}(x) \Rightarrow \mathbf{dyn}$

Shape Propagation (Formalization)

Static Shape

$$\text{static}(\llbracket \iota \rrbracket) = \mathbf{true}$$

$$\text{static}(\llbracket [\overline{s_i^n}] \rrbracket) = \bigwedge_i \text{static}(s_i)$$

$$\text{static}(\llbracket C(\overline{n_i : s_i}) \rrbracket) = \bigwedge_i \text{static}(s_i) \quad \text{static}(s_1 \cup s_2) = \text{static}(s_1) \wedge \text{static}(s_2)$$

$$\text{static}(\mathbf{dyn}) = \mathbf{false}$$

$$\text{static}(\perp) = \mathbf{false}$$

Remark. $\text{static}(s)$ returns true iff no subshape of s contains $\mathbf{dyn}$.

Shape Propagation (Pattern Matching)

Before giving the formal rules, let's give a simple trace of shape propagation for pattern matching.

```
class C(val n)
staged module Simple with
  fun f(x) =
    let y
    if x is C then y = x.n
    else y = 0
    y + 1
fun test()      = f(C(2))
fun test2(dyn) = f(C(dyn))
fun test3()     = f(0)
```

mls

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
fun f(x) = mls  
  let y  
    if x is C then  
      y = x.n  
    else  
      y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; `rest =  $\llbracket \perp \rrbracket$` so else is dead

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; `rest =  $\llbracket \perp \rrbracket$` so else is dead
3. In then: `sop(x.n) = sel( $\llbracket C(n: 2) \rrbracket$ , n) =  $\llbracket 2 \rrbracket$` , so $y \mapsto \llbracket 2 \rrbracket$

Shape Propagation (Pattern Matching)

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; `rest =  $\llbracket \perp \rrbracket$` so else is dead
3. In then: `sop(x.n) = sel( $\llbracket C(n: 2) \rrbracket$ , n) =  $\llbracket 2 \rrbracket$` , so $y \mapsto \llbracket 2 \rrbracket$
4. `y + 1` folds to $\llbracket 3 \rrbracket$, then Return 3

Cached as `f_C_Lit2`; body folds entirely to the constant 3.

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y: y` $\mapsto \llbracket \perp \rrbracket$

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
fun f(x) = mls  
  let y  
    if x is C then  
      y = x.n  
    else  
      y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
fun f(x) = mls
  let y
  if x is C then
    y = x.n
  else
    y = 0
  y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) = \llbracket \text{dyn} \rrbracket$

Shape Propagation (Pattern Matching)

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is `C`: $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) = \llbracket \text{dyn} \rrbracket$
4. $y + 1$: cannot fold ($\llbracket \text{dyn} \rrbracket + 1$); emit code, result $\llbracket \text{dyn} \rrbracket$

Cached as `f_C_Dyn`; body keeps the `y = x.n` assignment.

Shape Propagation (Pattern Matching)

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

ctx

$x \mapsto \llbracket 0 \rrbracket$

```
fun f(x) =
```

mls

```
  let y
```

```
    if x is C then
```

```
      y = x.n
```

```
    else
```

```
      y = 0
```

```
  y + 1
```

Shape Propagation (Pattern Matching)

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
fun f(x) =
```

mls

```
  let y
```

```
  if x is C then
```

```
    y = x.n
```

```
  else
```

```
    y = 0
```

```
  y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$

Shape Propagation (Pattern Matching)

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
fun f(x) = mls  
  let y  
    if x is C then  
      y = x.n  
    else  
      y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable

Shape Propagation (Pattern Matching)

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$

Shape Propagation (Pattern Matching)

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
fun f(x) = mls  
  let y  
  if x is C then  
    y = x.n  
  else  
    y = 0  
  y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$
4. $y + 1$ folds to $\llbracket 1 \rrbracket$, then Return 1

Cached as f_Lit0 .

Shape Propagation (Pattern Matching)

Resulting Cache

After all three calls, the staged module contains:

```
module Simple withs
  fun f_C_Dyn(x) =
    let y
      y = x.n
      y + 1          // dyn case: code is kept
    fun f_C_Lit2(x) = 3    // fully specialised
    fun f_Lit0()    = 1    // fully specialised
    fun test()      = 3
    fun test2(dyn)  = f_C_Dyn(C$If2(dyn))
    fun test3()     = 1
```

mls

Pattern Matching (Formalization)

Branch Filter (`filter`)

`filter` removes impossible shapes from the branches of a pattern match.

$$\text{filter}(s, _) = s$$

$$\text{filter}(\llbracket \iota_1 \rrbracket, \iota_2) = \llbracket \iota_1 \rrbracket \quad \text{if } \iota_1 = \iota_2$$

$$\text{filter}(\llbracket [\overline{s_i^n}] \rrbracket, \llbracket [^n] \rrbracket) = \llbracket [\overline{s_i^n}] \rrbracket$$

$$\text{filter}(\llbracket C(\overline{n_i : s_i^m}) \rrbracket, C) = \llbracket C(\overline{n_i : s_i^m}) \rrbracket$$

$$\text{filter}(\mathbf{dyn}, \pi) = \text{silh}(\pi)$$

$$\text{filter}(s_1 \cup s_2, \pi) = \text{filter}(s_1, \pi) \cup \text{filter}(s_2, \pi)$$

$$\text{filter}(s, \pi) = \perp \quad \text{otherwise}$$

Pattern Matching (Formalization)

Branch Remainder (rest)

rest removes shapes already covered by a match pattern.

$$\begin{aligned} \text{rest}(s, _) &= \perp & \text{rest}(\llbracket \iota_1 \rrbracket, \iota_2) &= \perp \quad \text{if } \iota_1 = \iota_2 \\ \text{rest}(\llbracket [\overline{s_i^n}] \rrbracket, \llbracket _{}^n \rrbracket) &= \perp & \text{rest}(\llbracket C(\overline{n_i : s_i^m}) \rrbracket, C) &= \perp \\ \text{rest}(\mathbf{dyn}, \pi) &= \mathbf{dyn} \quad \text{if } \pi \neq _ & \text{rest}(s_1 \cup s_2, \pi) &= \text{rest}(s_1, \pi) \cup \text{rest}(s_2, \pi) \\ \text{rest}(s, \pi) &= s \quad \text{otherwise} \end{aligned}$$

Pattern Matching (Formalization)

Dynamic Dispatching

When a method is called on a value with a **union shape**, we split by class and dispatch separately.

```
staged class B1(val y) with fun call(x) = x + 2 + y
staged class B2(val y) with fun call(x) = x + y
staged module M with
  fun twice(f, x) = f.call(f.call(x))
  fun pick(x, y, b) = if b then x else y
  fun f(b) =
    let m = pick(new B1(2), new B2(3), b)
    twice(m, 5)
```

Pattern Matching (Formalization)

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
fun f(b) =  
  let m = pick(new B1(2), new  
    B2(3), b)  
  twice(m, 5)
```

mls

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

Pattern Matching (Formalization)

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
fun f(b) =  
  let m = pick(new B1(2), new  
    B2(3), b)  
  twice(m, 5)
```

mls

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$

Pattern Matching (Formalization)

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
fun f(b) =  
  let m = pick(new B1(2), new  
    B2(3), b)  
  twice(m, 5)
```

mls

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$
2. Call $\text{pick}(\text{new } B1(2), \text{new } B2(3), b)$:
argument shapes are $\llbracket B1(2) \rrbracket$, $\llbracket B2(3) \rrbracket$,
 $\llbracket \text{dyn} \rrbracket$. Recurse into pick with a fresh ctx
 $\rightarrow$ go to Trace B

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =  
  if b then x  
  else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket dyn \rrbracket$

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =  
  if b then x  
  else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =  
  if b then x  
  else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =  
  if b then x  
  else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$

Pattern Matching (Formalization)

Trace B: `pick(B1(2), B2(3), dyn)`

```
fun pick(x, y, b) =  
  if b then x  
  else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$
4. Result shape: $\llbracket B1(2) \cup B2(3) \rrbracket$. Cached as pick1. Return to Trace A.

Pattern Matching (Formalization)

Trace A (continued): back in f

```
fun f(b) =
```

mls

```
  let m = pick(new B1(2), new  
              B2(3), b)
```

```
  twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

Pattern Matching (Formalization)

Trace A (continued): back in f

```
fun f(b) =  
  let m = pick(new B1(2), new  
    B2(3), b)  
  twice(m, 5)
```

mls

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape =
 $\llbracket B1(2) \cup B2(3) \rrbracket$

Pattern Matching (Formalization)

Trace A (continued): back in f

```
fun f(b) =  
  let m = pick(new B1(2), new  
    B2(3), b)  
  twice(m, 5)
```

mls

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape =
 $\llbracket B1(2) \cup B2(3) \rrbracket$
2. $\text{twice}(m, 5)$: m has **union shape** $\rightarrow$ go to **Trace C** and specialise as twice1

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =  
  let tmp  
  tmp = f.call(x)  
  f.call(tmp)
```

mls

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

Recall:

```
class B1(y) with  
  fun call(x) = x + 2 + y  
class B2(y) with  
  fun call(x) = x + y
```

mls

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =  
  let tmp  
  tmp = f.call(x)  
  f.call(tmp)
```

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \perp \rrbracket$

1. let tmp: extend ctx with $tmp \mapsto \llbracket \perp \rrbracket$

Recall:

```
class B1(y) with  
  fun call(x) = x + 2 + y  
class B2(y) with  
  fun call(x) = x + y
```

Pattern Matching (Formalization)

Trace C: `twice({B1(2), B2(3)}, 5)`

```
fun twice(f, x) =  
  let tmp  
  tmp = f.call(x)  
  f.call(tmp)
```

mls

Recall:

```
class B1(y) with  
  fun call(x) = x + 2 + y  
class B2(y) with  
  fun call(x) = x + y
```

mls

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \{9, 8\} \rrbracket$

1. let tmp: extend ctx with $tmp \mapsto \llbracket \perp \rrbracket$
2. `f.call(x)` on union $\rightarrow$ **split by class**:
 - $f = \llbracket B1(2) \rrbracket$: cache `f.call1()` $\rightarrow \llbracket 9 \rrbracket$
 - $f = \llbracket B2(3) \rrbracket$: cache `f.call1()` $\rightarrow \llbracket 8 \rrbracket$

Pattern Matching (Formalization)

Trace C (continued): second call

```
fun twice(f, x) =  
  let tmp  
  tmp = f.call(x)  
  f.call(tmp)
```

mls

Recall:

```
class B1(y) with fun  
call(x) = x + 2 + y  
class B2(y) with fun call(x) =  
x + y
```

mls

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \{9, 8\} \rrbracket$

3. $f.call(tmp)$ with tmp in $\llbracket \{9, 8\} \rrbracket$ (now $\llbracket dyn \rrbracket$): split again, body kept as code
 - $f = \llbracket B1 \rrbracket$: cache $f.call2(x) = x + 2 + 2$
 - $f = \llbracket B2 \rrbracket$: cache $f.call2(x) = x + 3$

Pattern Matching (Formalization)

Resulting Cache

After propagation, M and the staged classes contain:

```
module M with mls  
  fun f(b) =  
    let m, tmp1, tmp2  
    tmp1 = new B1(2)  
    tmp2 = new B2(3)  
    m = pick1(tmp1, tmp2, b)  
    twice1(m)  
  fun pick1(x, y, b) =  
    if b then new B1(2) else new B2(3)
```

```
fun twice1(f) =  
  let tmp  
  if f is  
    B1 then tmp = f.call1()  
    B2 then tmp = f.call1()  
  if f is  
    B1 then f.call2(tmp)  
    B2 then f.call2(tmp)
```

Pattern Matching (Formalization)

```
class B1(y) with mls  
  fun call1() = 9  
  fun call2(x) =  
    let tmp  
    tmp = x + 2  
    tmp + 2  
  
class B2(y) with  
  fun call1() = 8  
  fun call2(x) = x + 3
```

Printing Staged Block

After all the specialized functions are completed, we need to write the functions to a new file.

```
staged module M with
  val funCache = new Map([
    ["f1", ...],
    ["f2", ...],
    ["g1", ...]
  ])
```

mls

```
module M with
  fun f1(x, y) = ...
  fun f2() = ...
  fun g1() = ...
```

mls

Printing Staged Block

This is a similar process to staging, but done in reverse.

As before, we need extra care when handling symbols.

Motivation	1
Literature Survey	4
Overview	10
Staging	22
Shape Propagation	29
Benchmarking	58
Model-View-Projection transformation	59
Bibliography	60

Model-View-Projection transformation

basically we're pretty good at partially evaluating matrices w .

Time: about 2x (from 2s to 1s, eh...)

Code size: don't worry about it, we'll just do lifting

Bibliography

- [1] W. Taha, “A gentle introduction to multi-stage programming,” in *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, 2004, pp. 30–50.
- [2] A. Shali and W. R. Cook, “Hybrid partial evaluation,” in *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*, 2011, pp. 375–390.